

Image Processing Continued

You are not logged in.

Please [Log In](#) for full access to the web site.

Note that this link will take you to an external site (<https://shimmer.mit.edu>) to authenticate, and then you will be redirected back to this page.

Table of Contents

- [1\) Preparation](#)
- [2\) Reminders about Representations From Lab 1](#)
- [3\) Introduction](#)
- [4\) Correlation Revisited](#)
 - [4.1\) Edge Behaviors](#)
 - [4.1.1\) Extending](#)
 - [4.1.2\) Wrapping](#)
 - [4.1.3\) Implementation](#)
 - [4.2\) Check Your Results](#)
 - [4.3\) Blur With New Edge Behavior](#)
- [5\) Another New Filter](#)
 - [5.1\) Check Your Results](#)
- [6\) Color Images](#)
 - [6.1\) Loading and Saving Color Images](#)
- [7\) Filters on Color Images](#)
 - [7.1\) Check Your Results](#)
 - [7.2\) Other Kinds of Filters](#)
 - [7.3\) Check Your Results](#)
- [8\) Cascade of Filters](#)
 - [8.1\) Check Your Results](#)
- [9\) Seam Carving](#)
 - [9.1\) Outline and Description of Steps](#)
 - [9.1.1\) Graphical Depiction](#)
 - [9.2\) Suggested Helper Functions and Optional Tests](#)
 - [9.3\) Example with Intermediate Results](#)
 - [9.4\) Write It!](#)
 - [9.5\) Check Your Results](#)
- [10\) Something of Your Own](#)
 - [10.1\) Submitting This Part](#)
- [11\) Code Submission](#)
- [12\) Checkoff](#)

1) Preparation

This lab assumes you have Python 3.13 or later installed on your machine (3.14 recommended).

As with last week's lab, we will again use the `pillow` library for loading and saving images.

The following file contains code and other resources as a starting point for this lab:

[image_processing_2.zip](#)

Note that there is no `lab.py` file in this week's code distribution; you should start by copying last week's `lab.py` file into this week's code.

Issue with Last Week's Code

There were some issues with the code in last week's distribution for saving color images (which will be used later in this lab). After copying `lab.py` into the new distribution, replace `save_image` with the following:

```
def save_image(image, filename, buffer_filetype="PNG"):
    """
    Saves the given image to disk or to a file-like object. If
    filename is
        given as a string, the file type will be inferred from the
        given name. If
        filename is given as a file-like object, the file type will be
        determined
        by the "buffer_filetype" parameter.
    """
    # make sure we have a valid image
    if image['mode'] not in {'greyscale', 'color'}:
        raise ValueError(f'Unknown image mode: {image["mode"]}')
    if (pixcount := image['height'] * image['width']) !=
len(image['pixels']):
        raise ValueError(f'Incorrect number of pixels (expected
{pixcount}, got {len(image["pixels"])})')
    if image['mode'] == 'greyscale':
        if not all(isinstance(x, int) and 0 <= x <= 255 for x in
image['pixels']):
            msg = "Pixel values in a greyscale image must be
integers between 0 and 255"
            raise ValueError(msg)
    else:
        msg = "Pixel values in a color image must be tuples of
length 3"
        if not all(
            isinstance(p, tuple) and len(p) == 3
            and all(isinstance(x, int) and 0 <= x < 256 for x in
```

```

p)
    for p in image['pixels']
    ):
        msg = "Pixel values in a color image must be length-3
tuples of integers between 0 and 255"
        raise ValueError(msg)

    if isinstance(filename, str):
        if not os.path.isabs(filename):
            # treat relative paths as being relative to this file
            filename = os.path.join(os.path.dirname(__file__),
filename)

            # make folders if they do not exist
            directory =
os.path.realpath(os.path.dirname(filename))
            if directory and not os.path.exists(directory):
                os.makedirs(directory)

        # actually save image
        out = Image.new(
            mode=("L" if image["mode"] == "greyscale" else "RGB"),
            size=(image["width"], image["height"]),
        )
        out.putdata(image["pixels"])
        if isinstance(filename, str):
            out.save(filename)
        else:
            out.save(filename, buffer_filetype)
        out.close()

```

Most of your changes should be made to `lab.py`, which you will submit at the end of this lab. Importantly, you should not add any imports to the file, nor should you use the `pillow` module for anything other than loading and saving images (which are already implemented for you).

Your raw score for this lab will be reported out of 5 points. Your score for the lab is based on:

- correctly answering the questions on this page (2 points)
- passing the tests in `test.py` (2 points)
- a brief "checkoff" conversation with a staff member about your code (1 point)

All of the questions on this page, including your code submission, are due at 5:00pm on Friday, 13 February. Checkoffs are due at 10:00pm on Wednesday, 18 February. **It is a good idea to submit your lab early and often so that if something goes wrong near the deadline, we have a record of the work you**

completed before then. We recommend submitting your lab to the server after finishing each substantial portion of the lab.

Academic Integrity Policy

Please also review the [academic integrity policies](#) before continuing.

2) Reminders about Representations From Lab 1

In Lab 1, we introduced a representation for greyscale images, and we implemented some neat filters to operate on images of that form. In that lab, we limited our attention to greyscale images, where we represented brightness values as integers between 0 (the deepest black) and 255 (the brightest white).

Our filters were represented as Python functions that operated on images (i.e., functions that took images as input and produced related images as output).

Before we get into the core of this week's lab, answer the following question about the small piece of code below, which implements a small piece of functionality using our representations from last week.

```
def make_box(color):
    def create_image(h, w):
        return {
            "mode": "greyscale",
            "height": h,
            "width": w,
            "pixels": [color for _ in range(h * w)],
        }

    return create_image

maker = make_box(40)
im = maker(20, 30)
```

Which of the following are true statements about the code above?

- ☐ This code has a syntax error.
- ☐ This code does not have a syntax error, but Python will raise an exception when this code is run.
- ☐ This code will run without error.
- ☐ `make_box` is a function of a single argument.
- ☐ `make_box` is a function of two arguments.
- ☐ `make_box` is a dictionary.
- ☐ `maker` is a function of a single argument.
- ☐ `maker` is a function of two arguments.
- ☐ `maker` is a dictionary.
- ☐ `im` is a function of a single argument.
- ☐ `im` is a function of two arguments.
- ☐ `im` is a dictionary.

3) Introduction

In this lab, we'll continue building on the work from last week's lab. We'll start by expanding some of the ideas from last week, and then we'll add support for color images to our little image-processing library, as well as a few additional interesting filters and features.

4) Correlation Revisited

The main thing we implemented in last week's lab was *correlation*, which is a cool operation that can be used to implement a bunch of different effects. Last week, we saw two such effects: blurring and sharpening.

If we look at the output of our blurring filter from last week, though, maybe we see an opportunity for improvement. Here is the result of applying our blur filter to `cat.png` from last week:



Certainly this is kind of doing it's job: it took in an image of a cat and outputted a blurry version of that same image. But in addition to blurring the cat, we got a weird hazy frame around the edges of the image

because of how we were handling pixels around the edge of the image (remember that last week we treated any pixel values outside of the image as being 0).

4.1) Edge Behaviors

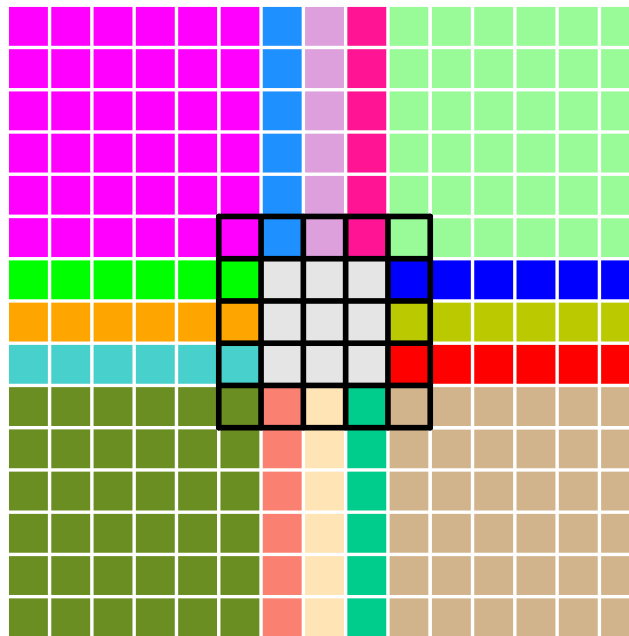
In this first section, we'll look at some alternative approaches for handling pixels outside of the bounds of the input image, and we'll see how we can use those behaviors to get better results out of our correlation operator.

We have several options for dealing with these edge effects:

- One option is to treat every out-of-bounds pixel as having a value of *zero*, which we already saw last week.
- Another is to *extend* the input image beyond its boundaries, explained in the next paragraph.
- Yet another is to *wrap* the input image at its edges, explained in the paragraph after that.

4.1.1) Extending

If we want to consider these out-of-bounds pixels in terms of an *extended* version of the input image, values to the left of the image should be considered to have the values from column 0, values to the top of the image should be considered to have the values from row 0, etc., as illustrated in the following diagram, where the bolded pixels in the center represent the original image, and the pixels outside that region represent the logical values in the "extended" version of the image (and note that this process continues infinitely in all directions, even though a finite number of pixels are shown below):



For example, in the grayscale image I with 3 rows and 2 columns below:

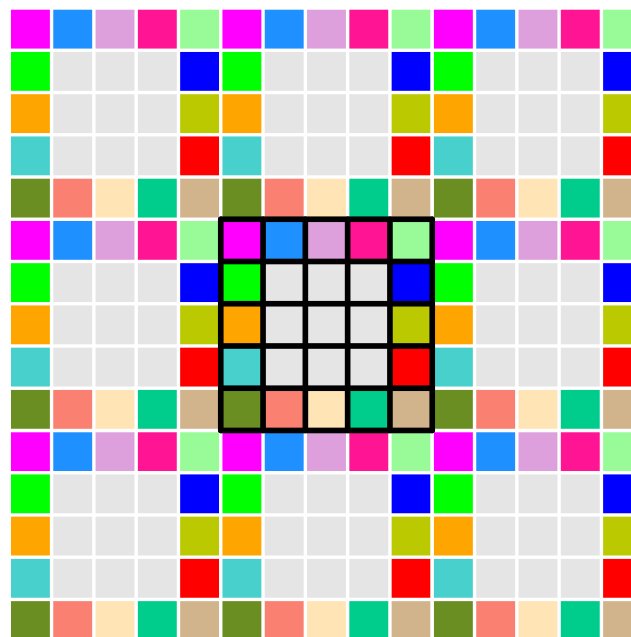
```
20  40
60  80
100 120
```

with the *extended* behavior, the value of the out-of-bounds pixel at row 1, column 2 is 80 and the value at row -1, column -1 is 20.

What would the value of the pixel at row 10, column 0 be in Image *I* with the *extended* behavior?
Enter a single number in the box below.

4.1.2) Wrapping

If we want to *wrap* the input image at its edges, that means trying to get values beyond the left edge of the image should return values from the right edge, but from the same row. Similarly, trying to get values beyond the top edge of the image should return values from the bottom edge, but from the same column. We can think of it as the image being infinitely 'tiled' in all four directions:



The thick black square above indicates the original image, and the copies are an illustration of the wrapping behavior (but they are not part of the actual image). Note that this only shows one copy of the image in each direction, but we would like the wrapping to extend infinitely in all directions.

For example, with the *wrap* behavior in the 3×2 image used previously, the value of the pixel at row 0, column -1 is 40, the value of the pixel at row -1, column 0 is 100, and the value of the pixel at row 3, column 2 is 20.

What would the value of the pixel at row 1, column 4 in Image I be with the *wrap* behavior? For reference, the greyscale image I with 3 rows and 2 columns is provided again here:

```
20  40
60  80
100 120
```

Enter a single number in the box below.

4.1.3) Implementation

Implement two new functions in your `lab.py` to handle these behaviors:

- `get_pixel_extend(image, r, c)` should return the value of pixel (r, c) in the input image, using the "extend" behavior discussed above for out-of-bounds pixels.
- `get_pixel_wrap(image, r, c)` should return the value of pixel (r, c) in the input image, using the "wrap" behavior discussed above for out-of-bounds pixels.

Finally, update your `correlate` method so that it optionally takes an additional argument: a function used for getting pixel values, which will allow us to control these edge behaviors. Adjust `correlate` so that it behaves as follows:

- `correlate(im, kernel, get_pixel)` should perform the same operation we saw last week (since `get_pixel` treats out-of-bounds pixels as zeros)
- `correlate(im, kernel, get_pixel_extend)` should perform correlation using the "extend" behavior for out-of-bounds pixels.
- `correlate(img, kernel, get_pixel_wrap)` should perform correlation using the "wrap" behavior for out-of-bounds pixels.
- `correlate(img, kernel)`, with no function explicitly specified, should use the "extend" behavior by default.

Note that your code for `correlate` should not condition on the input value; the difference in behaviors above should come about naturally as a result of the different behaviors implemented in the functions being passed in.

4.2) Check Your Results

When you have implemented your code and are confident that it is working, try running it on `test_images/pigbird.png` with the following kernel:


```
{(-4, -6): 0.5, (3, 2): 0.5}
```

Run it once with each edge behavior and save the results as PNG images. Take a look at those images. Do they match your expectations? How do the different edge-effect behaviors manifest in the output?

Note that you should call `round_and_clip_image` after calling `correlate` for each of these examples to make sure we end up with a valid image!

Result with 'zero' edge behavior:
No file selected

Result with 'extend' edge behavior:
No file selected

Result with 'wrap' edge behavior:
No file selected

4.3) Blur With New Edge Behavior

Note that the changes above should have made it so that, by default, your convolutions now use the "extend" edge behavior (implemented in `get_pixel_extend`). Try running your `blurred` function on `cat.png` again (run it with a box-blur kernel size of 13) and uploading the result:

New Blurry Cat Image:
No file selected

Much better! No more weird behavior on the edges of the image.

Now that we have that working, it's probably a good idea to go ahead and submit your code to the server at this point. Remember to **submit early and often!**

5) Another New Filter

Let's implement one last filter before moving on to color images. This one is a really neat filter called a [Sobel operator](#), which is useful for detecting edges in images.

This edge detector is a bit more complicated than the filters above because it involves *two* correlations¹. In particular, it involves kernels K_1 and K_2 , which are shown below:

K_1 :

```
-1 -2 -1
 0  0  0
 1  2  1
```

K_2 :

```
-1 0 1
-2 0 2
-1 0 1
```

After computing O_1 and O_2 by correlating the input with K_1 and K_2 , respectively (using the 'extend' behavior for each correlation), each pixel of the output O is the square root of the sum of squares of corresponding pixels in O_1 and O_2 :

$$O[r, c] = \text{round} \left(\sqrt{(O_1[r, c])^2 + (O_2[r, c])^2} \right)$$

As always, after performing this operation, take care to ensure that the final image is made up of integer pixels in range `[0, 255]`. But only clip the output *after* combining O_1 and O_2 . If you clip the intermediate results, the combining calculation will be incorrect.

Check Yourself:

What does each of the kernels K_1 and K_2 do, on its own? Try running, saving, and viewing the results of those intermediate correlations to get a sense of what is happening here.

Implement the edge detector as a function `edges ( image )`, which takes an image as input and returns a new image resulting from the above operations (where the edges should be emphasized). You should add this code to `lab.py` yourself (and make sure to include an informative docstring); there is no skeleton provided.

Note that you can compute square roots without need for an import by raising numbers to the power of 0.5.

5.1) Check Your Results

When you are done and your code passes the edge-detection tests, run your edge detector on the `test_images/construct.png` image, save the result as a PNG image, and upload it below:

Edges of `construct.png`:
No file selected

It's also a good idea to go ahead and submit your code to the server at this point. Remember to **submit**

early and often!

6) Color Images

For the rest of the lab, we'll shift our perspective a bit, moving to thinking about color images instead of greyscale images.

Although there are many ways we could choose to represent color images, we will use a tried-and-true representation, the [RGB color model](#), which is used in a lot of common image formats. In this representation, rather than a single integer, we will represent the color of each pixel as a tuple of three integers, representing the amount of red, green, and blue in the pixel, respectively. By combining red, green, and blue, we can represent a wide range of colors.

We'll represent color images in a very similar way to how we represented greyscale images.

- `"mode": "color"`
- `"width"`: the width of the image (in pixels),
- `"height"`: the height of the image (in pixels), and
- `"pixels"`: a Python list of pixel values, each represented as a tuple of three integers, `(r, g, b)`, where `r`, `g`, and `b` are all in the range `[0, 255]`. As with last week's lab, these values are stored in [row-major order](#) (listing the top row left-to-right, then the next row, and so on)

For example, consider this 3×2 (3 rows, 2 columns) image, enlarged here for clarity:



This image would be encoded as the following dictionary:

```
color_image = {  
    "mode": "color",  
    "height": 3,  
    "width": 2,  
    "pixels": [(255, 0, 0), (39, 143, 230),  
               (255, 191, 0), (0, 200, 0),  
               (100, 100, 100), (179, 0, 199)],  
}
```

6.1) Loading and Saving Color Images

The functions from last week's lab (`print_values`, `save_image`, and `load_image`) work for color images as well, but note that loading an image as a color image (rather than greyscale) requires explicitly specifying an argument:

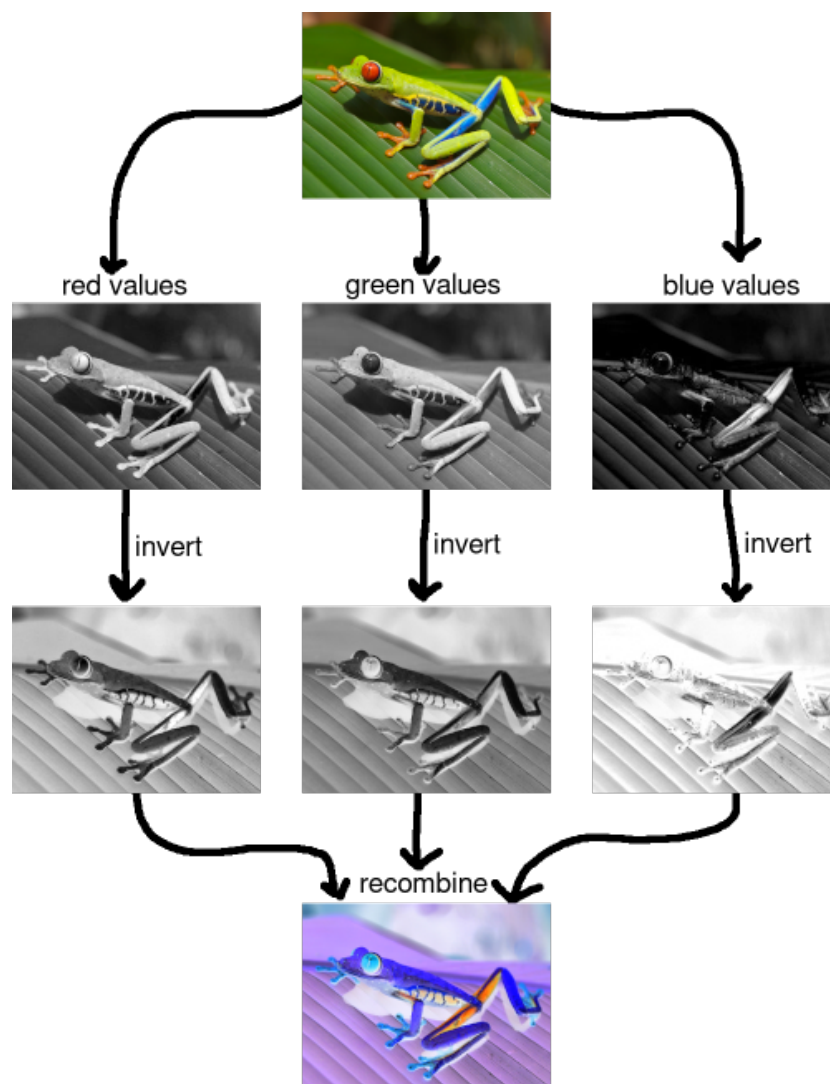
```
i = load_image("test_images/cat.png") # greyscale image
c = load_image("test_images/bluegill.png", mode="color") # color image
```

7) Filters on Color Images

In last week's lab, we used Python function objects to represent filters: we represented a filter as a function that took an image as input and produced a related image as output. We will continue this representation this week: a filter for color images is a Python function that takes a color image as its input and produces a related color image as its output.

While there are certainly other things we can do with color images, it turns out that we can think about implementing color versions of many of the filters from last week by separating our color image into three separate greyscale images (one for each channel: red, green, and blue), applying the same 'greyscale' filter from last week to each one, and then recombining the results together into a color image.

For example, we can think about a color version of our inversion filter as working in the way illustrated in the diagram below:



We can think about blurring or sharpening in similar ways: separate an image into its three color components, apply a filter to each, and recombine them together.

Given this common structure, it will be useful to have a common way to create these kinds of color image filters from their greyscale counterparts. As such, your first task for this lab is to implement a new function in `lab.py`, for which you can use the following skeleton as a starting point:

```
def color_filter_from_greyscale_filter(filt):
    """
    Given a filter that takes a greyscale image as input and produces a
    greyscale image as output, returns a function that takes a color
    image as
    input and produces the filtered color image, without modifying the
    input.
    """
    raise NotImplementedError
```

This function should take as input a *Python function* representing a greyscale filter (i.e., a function that takes a greyscale image as input and produces a greyscale image as its output). Its return value should be a *new function* representing a color image filter (i.e., a function that takes a color image as input and produces a color image as its output).

This new filter should follow the pattern above: it should split the given color image into its three components, apply the greyscale filter to each, and recombine them into a new color image.

An example of its usage is given below:

```
# if we have a greyscale filter called inverted that inverts a greyscale
# image...
inverted_grey_frog = inverted(load_image('grey_frog.png'))

# then the following will create a color version of that filter
color_inverted = color_filter_from_greyscale_filter(inverted)

# that can then be applied to color images to invert them (note that this
# should make a new color image, rather than mutating its input)
inverted_color_frog = color_inverted(load_image('color_frog.png',
mode='color'))
```

Implement the `color_filter_from_greyscale_filter` function in `lab.py`. You may find it helpful to define helper functions for splitting the color image into three separate greyscale images (one for each color component) and for recombining three greyscale images into a single new color image.

7.1) Check Your Results

Use this function to create a color version of your inversion filter from lab 1 and apply it to the `cat.png`

image from the `test_images` directory (make sure you've loaded it as a *color* image). Save your result as a color PNG image and upload your image below to be checked:

Inverted `cat.png`:
No file selected

It's also a good idea to go ahead and submit your code to the server at this point. Remember to **submit early and often!**

7.2) Other Kinds of Filters

Note that the structure of the `color_filter_from_greyscale_filter` function assumes a particular form for our greyscale filters: specifically, it expects that the filters we pass to it as inputs are functions of a *single argument*. However, some of our filters from last week did not follow this pattern. In particular, our `blurred` and `sharpened` filters took both an image and an additional parameter. So how can we implement these functions in such a way that they can work together with `color_filter_from_greyscale_filter`?

One strategy is to define, for example, a function `make_blur_filter` that takes the parameter `kernel_size` and returns a blur filter (which takes a single image as argument). In this way, we can make a blur filter that is consistent with the form expected by `color_filter_from_greyscale_filter`, for example:

```
# last week, we could create a blurred greyscale image like follows:
blurry = blurred(load_image('cat.png'), 3)

# we would like to make a function make_blur_filter that takes a single
# parameter and returns a filter. the use of this function is
# illustrated
# below:
blur_filter = make_blur_filter(3)
blurry2 = blur_filter(load_image('cat.png'))

# note that make_blur_filter(3), for example, produces a filter of the
# appropriate form for use with color_filter_from_greyscale_filter, but
# that blurry and blurry2 are equivalent images.

# it is also not necessary to store the output of make_blur_filter in a
# variable before calling it. for example, the following also produces
# an
# equivalent image:
blurry3 = make_blur_filter(3)(load_image('cat.png'))
```

Implement `make_blur_filter` and `make_sharpen_filter` in your `lab.py`. Each one of these functions should take a single argument `kernel_size` and should return a function appropriate for use with `color_filter_from_greyscale_filter`.

For the sake of avoiding repetitious code, you should try to make use of `blurred` and `sharpened` within your code, rather than reimplementing that behavior.

7.3) Check Your Results

Use these functions in conjunction with `color_filter_from_greyscale_filter` to produce the images described below and upload them to be checked:

Blurred `python.png`, with `kernel_size=9`:
No file selected

Sharpened `sparrowchick.png`, with `kernel_size=7`:
No file selected

It's also a good idea to go ahead and submit your code to the server at this point. Remember to **submit early and often!**

8) Cascade of Filters

Next, we'll add support for chaining filters together in a cascade (so that the output of one filter is used as the input to another, and so on). Add a new function to your `lab.py` for this behavior; a skeleton is provided below:

```
def filter_cascade(filters):  
    """  
    Given a list of filters, where each filter is a function that takes  
    an  
    image as input and produces an image as output, return a function  
    that  
    takes an image as input and produces an image that results from  
    applying  
    all of the individual filter effects in order.  
    """  
    raise NotImplementedError
```

This function should take a list of filters (Python functions) as input, and its output should be a function that, when called on an input image, produces the equivalent of applying each of the given filters, in turn, to the input image.

For example, if we have three filters `f1`, `f2`, and `f3`, then the following two images should be equivalent:

```
out1 = f3(f2(f1(image)))

c = filter_cascade([f1, f2, f3])
out2 = c(image)
```

Check Yourself:

Does it matter if `filters` contains color or greyscale filters? Can `filters` contain both color and greyscale filters?

Implement the `filter_cascade` function in your `lab.py`.

8.1) Check Your Results

When you are confident that your function is working as expected, apply the following filter to `test_images/frog.png`, save the result as a PNG image, and upload it below for checking:

```
filter1 = color_filter_from_greyscale_filter(edges)
filter2 = color_filter_from_greyscale_filter(make_blur_filter(5))
filt = filter_cascade([filter1, filter1, filter2, filter1])
```

`frog.png` filtered by the filter given above:
No file selected

It's also a good idea to go ahead and submit your code to the server at this point. Remember to **submit early and often!**

9) Seam Carving

You may wish to read all of the subsections in this part before implementing any code, so that you can [make a plan](#) before diving in.

Next, we'll implement a really neat kind of effect, often referred to as "content-aware resizing" or "retargeting." The goal of this technique is to scale down an image while preserving the perceptually important parts (e.g., removing background but preserving subjects).

Two common approaches for resizing an image are *cropping* and *scaling*. However, in certain situations, both of these methods can lead to undesirable results. The animation below shows these various kinds of resizing in action on an image of two cats², which is gradually scaled down from 300 pixels wide to 150

pixels wide. Cropping is shown on the first row, naive scaling is shown on the second row, and a form of content-aware resizing called *seam carving* (which we'll implement in this section) is shown on the third row.



The first two strategies have shortcomings: cropping causes one of the cats to be almost completely deleted, and scaling distorts both cats. By focusing on removing "low-energy" portions (defined below) of the image, however, the seam-carving technique manages to keep both cats almost completely intact while removing mostly background.

Here is another example, on a picture of several trees. Notice again that the trees are preserved almost exactly despite decreasing the width of the image by 75 pixels.



9.1) Outline and Description of Steps

The images above were generated using a technique called *seam carving*, which we'll implement in this

section. And, although the images above were the result of applying seam carving to greyscale images, your code should work for color images.

The idea behind seam carving is that, when we remove a pixel from each row to decrease the size of an image, instead of removing a straight vertical line, we instead find and remove connected "paths" of pixels from the top to the bottom of the image, with one pixel in each row. Each time we want to decrease the horizontal size of the image by one pixel, we start by finding the connected path from top to bottom that has the minimum total "energy," removing the pixels contained therein. To shrink the image further, we can apply this process repeatedly.

We will define the "energy" based on the edge-detection algorithm from last week's lab, applied to a greyscale version of the image: the energy of any given pixel will be the associated value in the output from edge detection, and the energy of a path is the sum total of the energies of the pixels it contains. The goal of finding and removing the minimum-energy path is accomplished by an algorithm which is outlined below³.

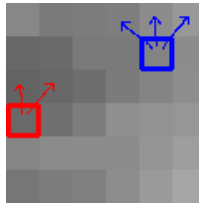
The algorithm works by repeating the following five steps until the width of the image reaches the desired size:

1. **Make a greyscale copy** of the current image, for using in computing the energy map. You can compute a color pixel's equivalent greyscale value v using the following formula, where r , g , and b represent the pixel's red, green, and blue values, respectively:
$$v = \text{round}(.299 \times r + .587 \times g + .114 \times b)$$
2. **Compute energy map.** Because we are using our edge detector as our "energy" function, this should simply involve calling the `edges` function on the greyscale image.
3. **Compute a "cumulative energy map"**, where the value at each position is the total energy of the lowest-energy path from the top of the image to that pixel.

For the top row, this will be equal to the top row from the energy map in step 2. Subsequent rows can be computed using the following algorithm (in pseudocode):

```
For each row of the "cumulative energy map":  
  For each pixel in the row:  
    Set this value in the "cumulative energy map" to be  
    the value of the pixel in the energy map, added to  
    the minimum "adjacent" value from the row above in the  
    cumulative energy map
```

For our purposes, we'll consider values in the row above from the cumulative energy map to be "adjacent" to the pixel we're considering if they are either directly above, or one pixel to the right or to the left of, the pixel location in question. For example, the pixel marked in blue below has three "adjacent" pixels in the row above (indicated by the arrows), and the pixel marked in red (on the edge of the image) has two "adjacent" pixels in the row above:



(After completing this process, the value at each pixel in this "cumulative energy map" will be the total energy of the lowest-energy path from the top of the image to that location.)

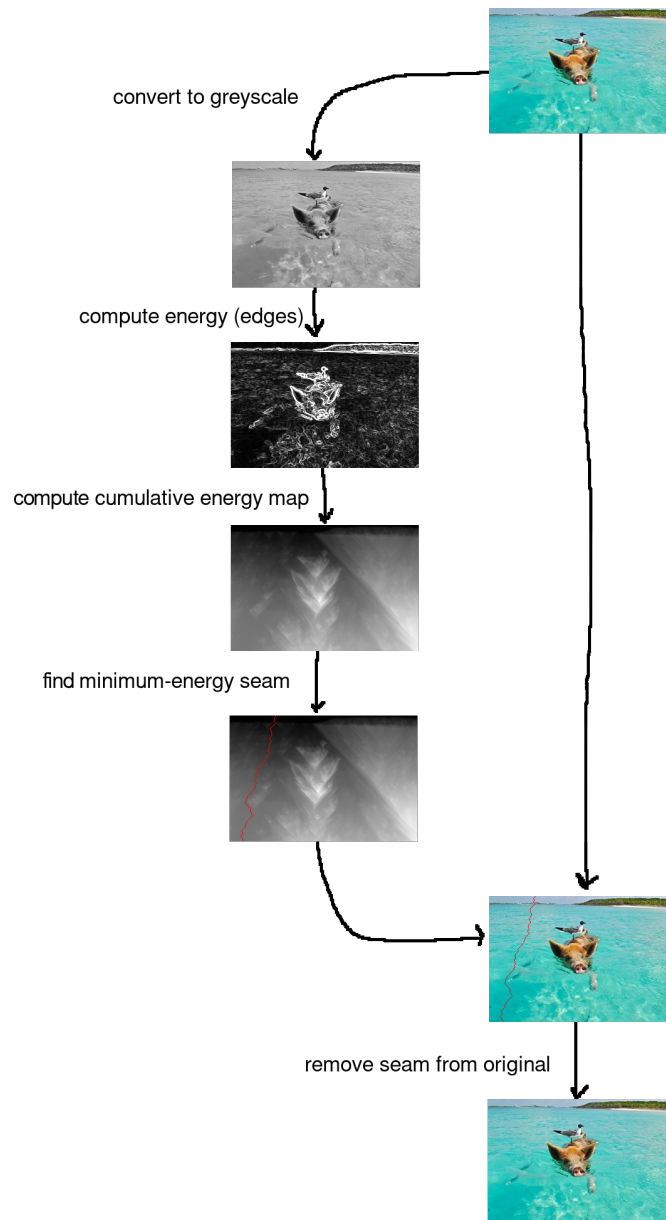
4. **Find the minimum-energy seam.** The minimum seam can then be found by tracing a path of adjacent pixels from the bottom to the top of the cumulative energy map. First, the minimum value pixel in the bottom row of the cumulative energy map is located. This is the bottom pixel of the minimum seam. The seam is then traced back up to the top row of the cumulative energy map by following the adjacent_pixels with the smallest cumulative energies.

Ties should always be broken by preferring the left-most of the tied columns.

5. **Remove the computed path.** Finally, we need to remove the pixels in the computed seam from our original color image (and we should use this smaller result when we start back at the top of these steps). Note, though, that we would like for this process to work without modifying the original image.

9.1.1) Graphical Depiction

The process described above (for removing one seam) is shown in the following diagram:



9.2) Suggested Helper Functions and Optional Tests

This is a pretty complicated process, so it will be helpful to break it down into smaller pieces.

While you are welcome to structure this code as you see fit, the end-to-end behavior should be implemented as the following function:

```
def seam_carving(color_image, num_cols):
    """
    Starting from the given color image, use the seam carving technique
    to
    remove num_cols (int) columns from the input image. Returns a new
    color image without modifying the original.
    """
    raise NotImplementedError
```

Because this is a pretty big problem, though, we recommend breaking it down into steps. You can break this down however you want, but here is one suggested set of helper functions:

- `greyscale_image_from_color_image(color_image)`: given a color image, return a greyscale image according to the formula above.
- `compute_energy(grey_image)`: given a greyscale image resulting from the step above, compute the energy map (using your `edges` function from above).
- `cumulative_energy_map(energy_image)`: given an energy map image resulting from the above step, return the "cumulative energy map" as described above.
- `minimum_energy_seam(cem)`: given a "cumulative energy map" dictionary, return a [set](#) of the indices into the pixels list associated with the vertical seam that should be removed from the original image.
- `image_without_seam(color_image, seam)`: given an image and a set of indices, return a new image with the associated pixels removed.

We have also provided test cases for these particular helper functions in the `test_seam_carving_helpers.py` file (which you can make use of in the same way as the normal `test.py`). The tests in `test_seam_carving_helpers.py` will *not* be checked on the server or included in your grade for this lab, but they can be helpful during debugging if you do choose to use the structure outlined above.

The optional test cases in `test_seam_carving_helpers.py` assume that each of these returns a structure like a greyscale image (a dictionary with `'mode'`, `'height'`, `'width'`, and `'pixels'` keys) but that these values are not necessarily valid images (i.e., the values in the `'pixels'` array for the cumulative energy map may be outside the `[0, 255]` range), except for `minimum_energy_seam`, which returns a set of integers.

9.3) Example with Intermediate Results

As you debug, it may also be helpful to have some concrete examples readily available in addition to the optional test cases provided in `test_seam_carving_helpers.py`. This section includes some intermediate results from removing one seam from `pattern.png`, shown enlarged below. While the exact structure of these results may be different (if you choose not to use the helper functions described above), the *values* contained in each should be the same regardless of your method.



The greyscale copy of `pattern.png` should look like the following:



Greyscale image with 4 rows and 9 columns:

```
200 160 160 160 153 160 160 160 200
200 160 160 160 153 160 160 160 200
0   153 160 160 160 160 160 153 0
0   153 153 160 160 160 153 153 0
```

Computing the energies (edges) produces:



Greyscale image with 4 rows and 9 columns:

```
160 160 0  28 0  28 0  160 160
255 218 10 22 14 22 10 218 255
255 255 30 0  14 0  30 255 255
255 255 31 22 0  22 31 255 255
```

Using this to compute the cumulative energy map yields the following (which looks very similar for this particular image but is actually different):



Greyscale image with 4 rows and 9 columns:

```
160 160 0  28 0  28 0  160 160
415 218 10 22 14 22 10 218 415
473 265 40 10 28 10 40 265 473
520 295 41 32 10 32 41 295 520
```

The minimum-energy seam is found by tracing the minimum value 10 in the bottom row, to the left adjacent 10s in the third and second row, and then taking the middle adjacent 0 in the top row. This seam is also indicated in red below:



and by the following set of indices (in any order):

```
{2, 11, 21, 31}
```

After removing the seam, the resulting image should look like `test_results/pattern_1seam.png`:



Color image with 4 rows and 8 columns:

```
(71 255 255) (114 200 71) (114 200 71) (100 200 50) (114 200 71) (114 200
71) (114 200 71) (71 255 255)
(71 255 255) (114 200 71) (114 200 71) (100 200 50) (114 200 71) (114 200
71) (114 200 71) (71 255 255)
(0 0 0) (100 200 50) (114 200 71) (114 200 71) (114 200 71) (114 200
71) (100 200 50) (0 0 0)
(0 0 0) (100 200 50) (100 200 50) (114 200 71) (114 200 71) (100 200
50) (100 200 50) (0 0 0)
```

9.4) Write It!

Fill in the body of the `seam_carving` function in your `lab.py` so that it implements seam carving using the method described above. Note that both the input and output should be *color* images, and note that the `seam_carving` function should not modify the image that it is given as input.

9.5) Check Your Results

When you are confident that things are working as expected, try running your code on `twocats.png`, removing 100 seams from it (reducing the width by 100). Save your results (in color), and upload the resulting files below for testing. Note that this may take a **long** time to run (up to a few minutes on some machines)! You may wish to add some print statements (for example, printing how many columns have been removed every time a new column is removed) to help keep track of your program's progress.

Seam Carved `twocats.png`:
No file selected

10) Something of Your Own

Finally, we would like for you to implement something new of your own choosing!

You are welcome to implement whatever you like here, and, unlike other portions of the lab, you are

welcome to import the `math` or `random` modules in this section. If you want to use other modules, that might be ok, but please ask us first via 6.101-help@mit.edu!

We encourage you to implement things that sound interesting to you! If you are having trouble thinking of what to do, here are some ideas that might help you get started, and we're certainly happy to answer questions as you work through things:

- You could implement some drawing primitives so that you can draw shapes on top of images. Maybe `circle(image, (r, g, b), x, y, radius)` draws a circle of the given size at the given location. You could implement multiple drawing primitives and use them to draw a nice new picture (or to modify an existing one!).
- You could implement some other new filters such as a [directional emboss](#). You could even combine them with some of the others using `filter_cascade` to make a new effect.
- You could add the equivalent of "pasting" smaller images on top of larger ones (maybe also including a color that should be treated as transparent for purposes of the copy/paste, so that pixels of that color in the pasted image are not copied over).
- You could implement a different kind of filter that takes spatial information into account, in addition to color information, such as a [vignette](#) or a [ripple filter](#).
- You could implement the equivalent of [color curves](#), for example by implementing something like described in [this article](#). Or, you could play around with the color content of images in other ways.
- You could implement the opposite of seam carving: smart resizing to *increase* the size of an image by inserting appropriate rows at low-energy regions in the image.
- You could hide a text or other data within an image using a technique called [steganography](#) and then implement a way to extract that data later on.

10.1) Submitting This Part

Regardless of what you choose to implement here, you should have some code that generates an image. Include your **documented** code for this part in `lab.py`, specifically in a **new** function called `custom_feature()`. This function should include all of the code that you needed to create your new custom feature, as well as code to generate an output image, and comments describing the change you implemented (including any additional imports that you used, if any).

Below are two boxes that you can use to upload images. If your custom feature applies a *filter* to an image (i.e., it operates on an input image), use the two boxes to upload an example of an input/output pair of images. If your custom feature simply generates an image (rather than operating on an existing image), you may leave the "Input image" box below empty.

Note: A green checkmark in this section only means that we have received your submission. You should be prepared to discuss your custom behavior during your checkoff.

Image before filter:

Input image (if any):
No file selected

Image after filter:

Image generated by your custom feature:
No file selected

11) Code Submission

When you have tested your code sufficiently on your own machine, submit your modified `lab.py` using the `6.101-submit` script. If you haven't already installed it, see the instructions on [this page](#).

The following command should submit the lab, assuming that the last argument `/path/to/lab.py` is replaced by the location of your `lab.py` file:

```
$ 6.101-submit -a image_processing_2 /path/to/lab.py
```

Running that script should submit your file to be checked. After submitting your file, information about the checking process can be found below:

When this page was loaded, you had not yet made any submissions.
If you make a submission, results should show up here automatically; or you may click [here](#) or reload the page to see updated results.

Please log in to see Hypothesis Profiler results.

12) Checkoff

Once you are finished with the code, you will need to come (in-person) to any open lab time and add yourself to the [queue](#) asking for a checkoff in order to receive credit for the lab. **You must be ready to discuss your code in detail before asking for a checkoff.**

You should be prepared to demonstrate your code (which should be well-commented, should avoid repetition, and should make good use of helper functions; see the notes on [style](#) for more information). In particular, be prepared to discuss:

- Your implementation of the different edge behaviors for correlation.
- Your implementation of edge detection.
- Your implementation of `color_filter_from_greyscale_filter`.
- Your implementation of `make_blur_filter`.
- Your implementation of `filter_cascade`.

- Your implementation of the seam-carving.
- Your creative extension.

You have not yet received this checkoff. When you have completed this checkoff, you will see a grade here.

Footnotes

¹ It is for this reason, and specifically since we want to compute these correlations *without rounding* before combining the results together, that we suggested separating `correlate` and `round_and_clip_image` rather than implementing a single function that performed both operations.

² This animation is licensed under a [CC BY SA 4.0 International License](#). The original image of two cats, which is licensed under a [CC BY 2.0 License](#), comes from [wellflat](#) on Flickr.

³ For more information, see the [Wikipedia page for seam carving](#).